Politechnika Wrocławska
Wydział Informatyki i Telekomunikacji

Zarządzanie w systemach i sieciach komputerowych - projekt

Mechanizmy zarządzania wątkami programowymi w zadaniu wykrywania krawędzi na obrazach cyfrowych z wykorzystaniem operatorów gradientowych

Autorzy:

Karol Gąsior
Indeks: 280913

Mateusz Kunicki
Indeks: 272468

Prowadzący zajęcia:

Dr inż. Robert Wójcik, W4N

Ocena pracy:

Wrocław, 2026

1 Temat i cel projektu

Temat: Mechanizmy zarządzania wątkami programowymi w zadaniu wykrywania krawędzi na obrazach cyfrowych z wykorzystaniem operatorów gradientowych (Sobela i pokrewnych).

Cel projektu: projekt oraz implementacja programu wykrywającego krawędzie na obrazie wczytywanym z dysku, w którym obliczenia rozdzielane są pomiędzy dowolną, zadaną przez użytkownika liczbę wątków. Za podział pracy odpowiadać będzie wbudowany mechanizm szeregujący (*scheduler*) oferujący kilka strategii przydziału fragmentów obrazu. Dodatkowym celem jest zbadanie, jak liczba wątków, strategia szeregowania oraz rozmiar przydzielanego fragmentu wpływają na czas przetwarzania (zegarowy i procesora) oraz na zużycie pamięci operacyjnej.

2 Sformułowanie problemu

Problem wykrywania krawędzi będzie rozwiązywany przy uwzględnieniu następujących założeń:

- obraz wejściowy (`input.jpg`) wczytywany jest z dysku i od razu konwertowany do skali szarości (8 bitów na piksel);
- cały obraz mieści się w pamięci operacyjnej i jest przetwarzany w niej w całości;
- użytkownik wybiera jedną z dostępnych masek: Sobel 3×3, Sobel 5×5, Prewitt 3×3, Scharr 3×3 oraz krzyż Robertsa [1, 2];
- piksele leżące poza brzegiem obrazu zastępowane są najbliższym pikselem brzegowym (powielenie krawędzi);
- wynikiem jest moduł gradientu znormalizowany do zakresu 0–255 i zapisany do pliku `output.jpg`;
- najmniejszą jednostką podziału pracy pomiędzy wątki jest wiersz obrazu;
- liczba wątków, strategia szeregowania i rozmiar fragmentu są parametrami programu;
- program działa wyłącznie w trybie tekstowym (terminal), bez interfejsu graficznego.

Problem obliczeniowy [1, 2]: dla obrazu I o wymiarach $W \times H$ oraz pary masek K_x, K_y o rozmiarze $k \times k$ wyznaczyć dla każdego piksela wartość

$$G(x,y) = \sqrt{G_x(x,y)^2 + G_y(x,y)^2}, \quad G_x = K_x * I, \quad G_y = K_y * I, \tag{1}$$

gdzie $*$ oznacza dwuwymiarowy splot dyskretny.

Problem badawczy (optymalizacyjny) [3, 4]: dobrać liczbę wątków N, strategię szeregowania oraz rozmiar fragmentu tak, aby zapewnić optymalne wartości następujących kryteriów:

- czas zegarowy przetwarzania obrazu – minimum;
- efektywność zrównoleglenia $E(N) = S(N)/N$, gdzie $S(N) = T(1)/T(N)$ – maksimum;
- nierównomierność obciążenia wątków (różnica czasów zakończenia pracy) – minimum;
- dodatkowe zużycie pamięci RAM związane z wielowątkowością – minimum.

3 Analiza złożoności obliczeniowej problemu

Złożoność obliczeniowa splotu wynosi $O(W \cdot H \cdot k^2)$, a więc dla ustalonej maski zależy liniowo od liczby pikseli obrazu. Złożoność pamięciowa wynosi $O(W \cdot H)$ – program przechowuje obraz wejściowy (1 bajt na piksel), mapę modułu gradientu (4 bajty na piksel) oraz obraz wyjściowy (1 bajt na piksel), czyli około 6 bajtów na piksel.

Wartość każdego piksela wyniku zależy wyłącznie od jego otoczenia w obrazie wejściowym, dlatego zadanie należy do klasy problemów łatwo zrównoleglanych (*embarrassingly parallel*) i teoretycznie umożliwia przyspieszenie liniowe względem liczby rdzeni. W praktyce przyspieszenie ograniczają: część sekwencyjna programu (dekodowanie i kodowanie JPG, redukcja maksimum), narzut synchronizacji oraz przepustowość pamięci. Górne ograniczenie przyspieszenia opisuje prawo Amdahla [5]:

$$S(N) \leq \frac{1}{(1-p) + \dfrac{p}{N}}, \tag{2}$$

gdzie p oznacza udział części programu, którą można wykonywać równolegle.

4 Metoda i algorytmy rozwiązywania problemu

Zastosowana zostanie dekompozycja danych: obraz dzielony jest na fragmenty złożone z kolejnych wierszy, a każdy wątek przetwarza otrzymane fragmenty niezależnie, zapisując wyniki do rozłącznych obszarów pamięci, co eliminuje potrzebę stosowania blokad. Przetwarzanie przebiega w dwóch etapach rozdzielonych barierą:

1. splot z maską oraz wyznaczenie lokalnego maksimum gradientu w każdym wątku;
2. redukcja lokalnych maksimów do maksimum globalnego, a następnie równoległa normalizacja wyniku do zakresu 0–255.

Wątki tworzone są jednokrotnie i utrzymywane w puli (*thread pool*), dzięki czemu kolejne etapy nie ponoszą kosztu ich ponownego tworzenia. Przydziałem fragmentów obrazu zarządza scheduler oferujący strategie wzorowane na klauzuli *schedule* standardu OpenMP [4]:

- *static* – obraz dzielony na N równych bloków, po jednym na wątek;
- *cyclic* – fragmenty o stałej wielkości przydzielane cyklicznie (round-robin);
- *dynamic* – wolny wątek pobiera kolejny fragment ze wspólnej kolejki (licznik atomowy);
- *guided* – jak *dynamic*, lecz rozmiar fragmentów maleje wraz z ilością pozostałej pracy.

5 Metoda, technologie i narzędzia implementacji

Program zostanie napisany w języku C++ (standard C++17) z wykorzystaniem biblioteki STL, w szczególności mechanizmów wielowątkowości: `std::thread`, `std::mutex`, `std::condition_variable` oraz `std::atomic` [3, 6]. Odczyt i zapis plików JPG zrealizowany zostanie przy użyciu jednoplikowych bibliotek `stb_image` i `stb_image_write` [7]. Projekt będzie budowany za pomocą CMake i realizowany w środowisku Qt Creator (bez wykorzystania bibliotek Qt). Logika przetwarzania zostanie zamknięta w klasie

`EdgeDetector`, której parametrami są liczba wątków, strategia szeregowania oraz rozmiar fragmentu.

6 Sposób testowania i oceny jakości rozwiązań

Poprawność działania zostanie zweryfikowana poprzez porównanie wyników wersji wielowątkowej z wynikiem wersji jednowątkowej – obrazy wyjściowe muszą być identyczne niezależnie od liczby wątków i strategii szeregowania. Testy przeprowadzone zostaną na obrazach syntetycznych o znanym położeniu krawędzi oraz na zdjęciach rzeczywistych.

Ocena wydajności obejmie pomiar dla każdego etapu (wczytanie, splot, normalizacja, zapis): czasu zegarowego – przy użyciu biblioteki `chrono` STL [6], czasu procesora całego procesu oraz poszczególnych wątków, a także bieżącego i szczytowego zużycia pamięci RAM (funkcje systemowe Windows i Linux). Wbudowany tryb porównawczy wykona przetwarzanie dla $1, 2, 4, \ldots, N$ wątków z kilkukrotnym powtórzeniem pomiaru i wyznaczy przyspieszenie oraz efektywność. Badania zostaną przeprowadzone dla obrazów o różnej rozdzielczości (od HD do 8K), różnych masek, strategii szeregowania i rozmiarów fragmentu.

W aplikacji nie przewiduje się interfejsu graficznego. Wyniki pomiarów będą wypisywane w terminalu oraz zapisywane do plików tekstowych, a następnie analizowane. W celu wizualnej prezentacji wyników badań przedstawione zostaną odpowiednie wykresy.

Literatura

[1] Gonzalez R. C., Woods R. E., *Digital Image Processing*, 4th ed., Pearson, 2018.

[2] Tadeusiewicz R., Korohoda P., *Komputerowa analiza i przetwarzanie obrazów*, Wydawnictwo Fundacji Postępu Telekomunikacji, Kraków, 1997.

[3] Williams A., *C++ Concurrency in Action*, 2nd ed., Manning, 2019.

[4] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, 2021.

[5] Amdahl G. M., *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS Conference Proceedings, vol. 30, 1967.

[6] `https://en.cppreference.com/w/cpp/thread`, 19.10.2026.

[7] `https://github.com/nothings/stb`, 19.10.2026.